Optimising Numerical Code

Suman 5 Feb, 2026 At 8:00 PM Pre-Reads Recommended Resource

Associated Lectures (1)

Description Discussions

Pre-Read: Optimising Numerical Code

1. What You'll Learn

In this pre-read, you will discover the secrets to making your Python code run lightning-fast. By the end of this reading, you will:

- **Discover** why traditional loops can slow down your programs and how to spot them.
- **Understand** the power of "Vectorization"—doing math on thousands of numbers at once.
- **Learn** about special tools like Numba that act as a "turbo button" for your code.
- **Visualize** the difference between writing instructions for a human vs. instructions for a machine.

---

2. Detailed Explanation

A. Introduction: What Is Code Optimization?

Imagine you are at a grocery store checkout.

- **The Slow Way (Loops):** You put one item on the belt. The cashier scans it. You put the next item. They scan it. This is repeated for 50 items. It takes a long time.
- **The Optimized Way (Vectorization):** You have a magic wand that scans your entire cart instantly in one second.

Code Optimization is the process of rewriting your code so it runs faster and uses less memory, often by removing manual repetition.

In technical terms, we move away from doing things one by one (iteration) and move toward doing things in batches (vectorization).

B. Importance: Why Does This Matter?

Python is famous for being easy to write, but it is also famous for being "slow" compared to other languages if you write it the wrong way. Here is why optimization matters to you:

Time is Precious: In data science, you might process millions of rows of data. A slow loop could take 2 hours to run. Optimized code might finish the same task in 10 seconds.

Real-Time Results: If you are building an app or a game, the computer needs to calculate physics or graphics instantly. Slow code makes apps "laggy."

Battery and Cost: Slow code makes the computer work harder. This drains your laptop battery faster and costs more money if you are renting cloud servers.

C. Building Understanding: From Known to New

Let's look at a classic problem: **Squaring a list of numbers.**

The Painful Way (Python Loops) You have a list of numbers, and you want to square each one. You naturally reach for a `for` loop.

```
# The "Slow" Way
numbers = [1, 2, 3, 4, 5]
squared_numbers = []

# We visit every number one by one
for number in numbers:
    squared_numbers.append(number ** 2)

print(squared_numbers)
# Output: [1, 4, 9, 16, 25]
```

Why is this slow? Python has to check every single item, ask "what is this?", "what do I do with it?", and then calculate. It adds a tiny delay for every number. If you have 1 million numbers, those tiny delays add up to a huge delay.

The Solution (Vectorization) Instead of a loop, we use NumPy. We treat the entire list as a single object.

```
import numpy as np

# The "Fast" Way
numbers_array = np.array([1, 2, 3, 4, 5])

# We tell the computer: "Square everything in this array."
squared_array = numbers_array ** 2

print(squared_array)
# Output: [ 1  4  9 16 25]
```

Why is this fast? NumPy skips the "checking" phase. It knows every item is a number, so it sends a command to the computer processor to square them all simultaneously.

D. Core Components

To optimize code, we rely on three main concepts.

1. Vectorization

- **Concept**: Replacing explicit loops (doing things one by one) with array expressions (doing things in batches).
- *Example*: Instead of `a + b` inside a loop, we just do `array_A + array_B`.

2. Broadcasting

- **Concept**: A set of rules that allows NumPy to do math on arrays of different shapes without making unnecessary copies of data.
- *Example*: If you want to add `5` to a million numbers, Broadcasting applies that `5` to everyone virtually, without creating a list of a million `5`s.

3. Compilation (The "Turbo Button")

- **Concept**: Tools like **Numba** or **Cython**. These translate your easy-to-read Python code into machine-code (0s and 1s) that the computer understands natively.
- *Example*: You add a special tag to your function, and Python translates it into a super-fast version before running it.

E. Step-by-Step Process: How to Optimize

When you have slow code, don't guess. Follow these steps:

Step 1: Profile (Measure) Don't optimize everything. Find the specific part of your code that is slow. We call this the "bottleneck."

> *Analogy*: If your car is slow, check if the tire is flat before replacing the engine.

Step 2: Vectorize (Remove Loops) Look for `for` loops that do math. Can you replace them with a NumPy operation?

> *Action*: Turn `for i in list: x = i + 1` into `array + 1`.

Step 3: Compile (Advanced) If it is still too slow, use a tool like Numba.

> *Action*: Import Numba and tag your function (we will see this in the lecture!).

F. Key Features: The "Turbo" Tools

In the lecture, we will briefly look at two advanced tools. You don't need to master them yet, just know they exist.

1. Cython This is a mix of C (a very fast, old programming language) and Python. It allows you to give Python explicit clues about data types (like "this variable is definitely an integer").

- *Benefit*: Extremely fast.
- *Drawback*: Harder to write.

2. Numba This is a "Just-In-Time" (JIT) compiler. It watches your Python function and, right before it runs, converts it to fast machine code.

- *Benefit*: Very easy to use. Often just one line of extra code!
- *Drawback*: Doesn't work on *every* kind of Python code, mostly just math.

G. Putting It All Together

Let's look at a comparison of calculating the **hypotenuse** of a triangle (using Pythagoras' theorem:) for thousands of triangles.

Scenario: We have two lists, `side_a` and `side_b`, each containing 10,000 numbers.

```
import numpy as np
import math

# 1. The Slow Loop (Python)
def pythagoras_loop(a_list, b_list):
    result = []
    # We loop 10,000 times. Very slow.
    for i in range(len(a_list)):
        c = math.sqrt(a_list[i]**2 + b_list[i]**2)
        result.append(c)
    return result

# 2. The Vectorized Way (NumPy)
def pythagoras_vector(a_array, b_array):
    # We do it all at once. Fast.
    return np.sqrt(a_array**2 + b_array**2)

# 3. The Numba Way (The Turbo)
from numba import jit

@jit(nopython=True) # This is the magic tag
def pythagoras_numba(a_array, b_array):
    # It looks like a loop, but Numba compiles it to machine code!
    result = np.empty_like(a_array)
    for i in range(len(a_array)):
        result[i] = math.sqrt(a_array[i]**2 + b_array[i]**2)
    return result
```

In the lecture, we will race these three against each other. **Spoiler Alert**: The Vectorized version is usually 50x faster, and Numba can be even faster!

---

3. Practice Exercises

Get your brain ready for the lecture by thinking through these scenarios.

Exercise 1: Pattern Recognition

Look at the code below. Is this efficient or inefficient? Why?

```
prices = [10, 20, 30, 40]
discounted_prices = []
discount = 0.9

for price in prices:
    new_price = price * discount
    discounted_prices.append(new_price)
```

Exercise 2: Concept Detective

You are working on a weather app. You have the temperature for every city in the world (10,000 cities). You want to convert all of them from Celsius to Fahrenheit.

- **Option A**: Write a function that converts one city, and loop it 10,000 times.
- **Option B**: Store the temperatures in a NumPy array and apply the formula to the whole array.

Which option is "Vectorized"?

Exercise 3: Real-Life Application

Think of a situation in a video game where thousands of calculations happen at the same time. *(Hint: Think about explosions, particles, or moving crowds.)*

Ready for the lecture? You now have the vocabulary: Vectorization, Loops, and Compilation. In the live session, we will measure the exact speed of these methods and you will see the "Speedometer" jump from slow to supersonic!